

Odločitvena drevesa in pravila

Ljupčo Todorovski

Univerza v Ljubljani, Fakulteta za matematiko in fiziko
Institut Jožef Stefan, Odsek za tehnologije znanja (E8)

Marec 2026

- 1 Odločitvena drevesa, *Decision trees*
 - Uvod in primeri
 - Učenje dreves
 - Primer učenja
 - Rezanje dreves
 - Kontrola pristranskosti in variance
- 2 Odločitvena pravila
 - Uvod in primeri
 - Učenje pravil
 - Primer učenja

Zgradba odločitvenega drevesa

Notranje vozlišče

Testira vrednost napovedne spremenljivke X , npr. $X < 0.5$ ali $X = m$.

Veji iz notranjega vozlišča

Ustrezata izidom testa, *True* in *False*.

Končno vozlišče (list)

Napove vrednost ciljne spremenljivke Y , npr. $\hat{Y} = 0.4$ ali $\hat{Y} = \oplus$.

Kaj predstavlja odločitveno drevo?

Razbije prostor $D_{\mathbf{X}} = D_{X_1} \times D_{X_2} \times \cdots \times D_{X_p}$ na k disjunktnih regij

$$D_{\mathbf{X}} = \bigcup_{i=1}^k R_i$$

$$\forall i, j \in \{1, 2, \dots, k\}, i \neq j: R_i \cap R_j = \emptyset$$

Vsaka regija R_i ustreza enemu listu drevesa

- Test v notranjem vozlišču razbije $D_{\mathbf{X}}$ na disjunktni podmnožici
- Ena podmnožica ustreza izidu testa (veji) *True*, druga veji *False*
- Presek podmnožic na vejah od korena do lista je regija tega lista

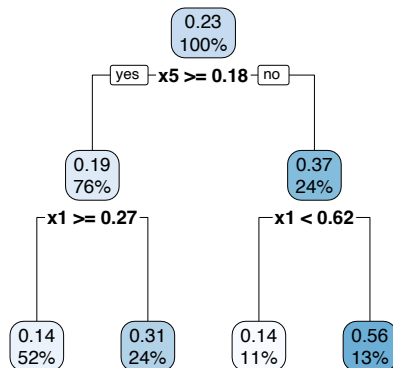
Kaj predstavlja odločitveno drevo?

Odločitveno drevo definira odsekoma linearno funkcijo

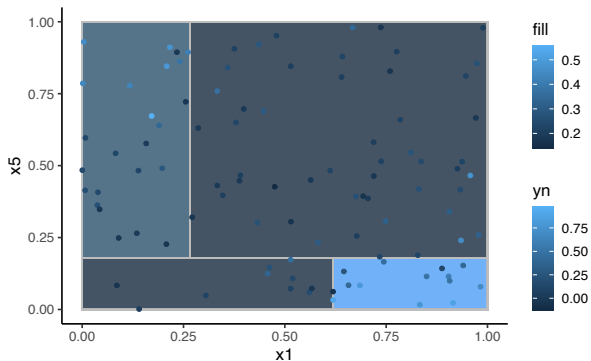
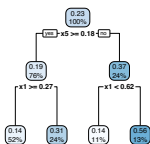
$$m: D_{X_1} \times D_{X_2} \times \cdots \times D_{X_p} \rightarrow D_Y$$
$$m(\mathbf{x}) = \sum_{i=1}^k \mathbb{1}(\mathbf{x} \in R_i) c_i$$

- Vrednost $c_i \in D_Y$ je napoved ciljne spremenljivke v listu regije R_i
- Ker so regije disjunktne, vsak $\mathbf{x}$ pripada točno eni regiji R_x
- Zato je v zgornji vsoti zgolj en sumand različen od 0
- Napoved je torej enaka c_x , t.j., napovedi lista, ki ustreza regiji R_x

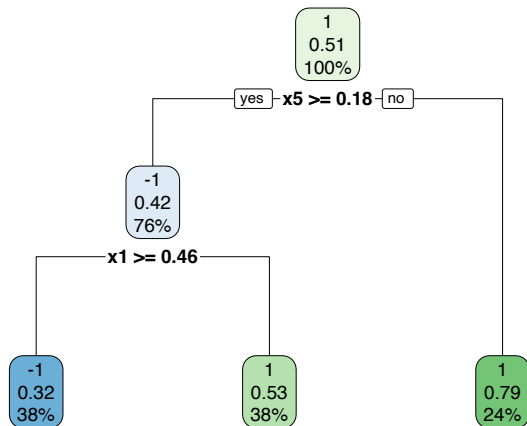
Primer regresijskega drevesa



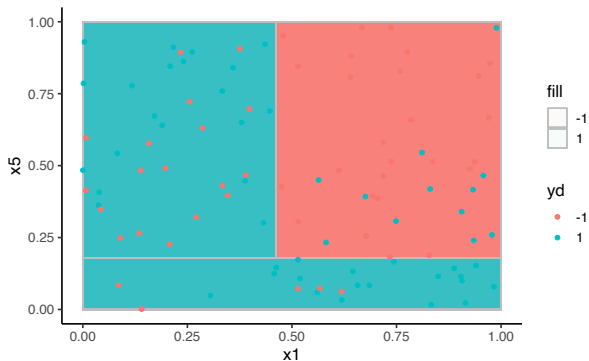
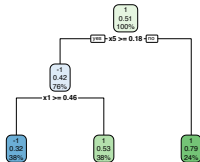
Napovedi regresijskega drevesa



Primer klasifikacijskega drevesa



Napovedi klasifikacijskega drevesa



Napovedovanje z odločitvenim drevesom

Require: T je odločitveno drevo

Require: $\mathbf{x}$ je vektor vrednosti napovednih spremenljivk

Ensure: $\hat{y}$ je napovedana vrednost ciljne spremenljivke

```
function napoved( $T$ ,  $\mathbf{x}$ )  
  if list( $T$ ) then  
    return  $\hat{y} = T$ .napoved  
  end if  
  if  $T$ .test( $\mathbf{x}$ ) then  
    return napoved( $T$ .levo,  $\mathbf{x}$ )  
  else  
    return napoved( $T$ .desno,  $\mathbf{x}$ )  
  end if  
end function
```

Vozlišča drevesa n ustrezajo množicam učnih primerov $S(n)$

Korenu drevesa n_{root} ustreza množica vseh učnih primerov S , $S(n_{root}) = S$.

Za notranje vozlišče n s potomcema n_D in n_L velja

$$S(n_L) = \{e \in S(n) : n.test(e) = True\}$$

$$S(n_D) = \{e \in S(n) : n.test(e) = False\}$$

$$S(n_L) \cap S(n_D) = \emptyset$$

$$S(n) = S(n_L) \cup S(n_D)$$

Napoved lista n iz učnih primerov $S(n)$

Za regresijo je to povprečna vrednost Y v $S(n)$

$$\text{napoved}(n) = \frac{1}{|S(n)|} \sum_{\langle \mathbf{x}, y \rangle \in S(n)} y$$

Za klasifikacijo je to modus Y v $S(n)$

- Primerjamo moči množic $S_i = \{\langle \mathbf{x}, y \rangle \in S(n) : y = y_i\}$
- Napovemo y_i pri katerem ima množica S_i največjo moč
- Napovemo lahko tudi verjetnost vrednosti y_i ocenjeno kot $|S_i|/|S|$

Funkcija nečistoče podatkovne množice

$$imp: 2^S \rightarrow \mathbb{R}$$

Za podano podatkovno množico $T \subseteq S$

- Izračuna nečistočo oz. varianco vrednosti ciljne spremenljivke Y v T
- Za numerično ciljno spremenljivko je varianca jasno definirana
- Kako izračunamo varianco za diskretne spremenljivke?

Nečistoča numerične ciljne spremenljivke

$$\text{imp}(T) = \frac{1}{|T|} \sum_{\langle \mathbf{x}, y \rangle \in T} (y - \bar{y})^2$$
$$\bar{y} = \frac{1}{|T|} \sum_{\langle \mathbf{x}, y \rangle \in T} y$$

Diskretna ciljna spremenljivka

$$\begin{pmatrix} y_1 & y_2 & \dots & y_c \\ p_1 & p_2 & \dots & p_c \end{pmatrix}$$

To je porazdelitev vrednosti $\{y_1, y_2, \dots, y_c\}$ ciljne spr. Y v primerih iz T

$$T_i = \{e = \langle \mathbf{x}, y \rangle \in T : y = y_i\}$$
$$p_i = P(Y = y_i) = \frac{|T_i|}{|T|}, i = 1, 2, \dots, c$$

Nečistoča diskretne ciljne spremenljivke

$$\text{imp}(T) = \phi(p_1, p_2, \dots, p_c)$$

Zaželenne lastnosti funkcije ϕ

- 1 Doseže maksimalno vrednost pri enakomerni porazdelitvi $\forall i: p_i = 1/c$
- 2 Doseže minimalno vrednost pri porazdelitvah, kjer $\exists i: p_i = 1$
- 3 Simetrična: neobčutljiva na vrstni red parametrov

Količina informacije (entropija) in nečistoča

$$\phi(p_1, p_2, \dots, p_c) = - \sum_{i=1}^c p_i \log_2 p_i$$

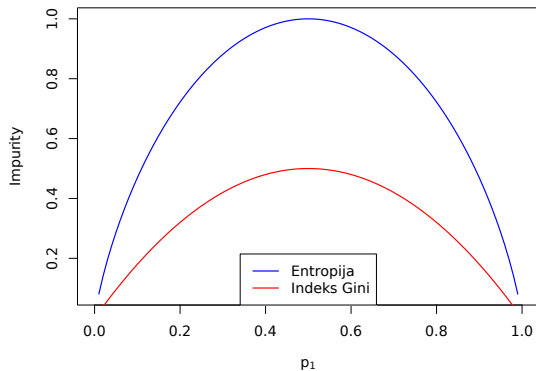
- 1 Maksimalna vrednost je $\log_2 c$ pri $p_i = 1/c$
- 2 Minimalna vrednost je 0 pri $\exists! i: p_i = 1$
- 3 Vsota je neodvisna od vrstnega reda sumandov

Indeks Gini in nečistoča

$$\phi(p_1, p_2, \dots, p_c) = 1 - \sum_{i=1}^c p_i^2$$

- ❶ Maksimalna vrednost je $1 - 1/c$ pri $p_i = 1/c$
- ❷ Minimalna vrednost je 0 pri $\exists! i: p_i = 1$
- ❸ Vsota je neodvisna od vrstnega reda sumandov

Primerjava obeh funkcij pri $c = 2$



Učenje odločitvenega drevesa

Require: S je množica učnih primerov

Ensure: T je naučeno odločitveno drevo

```
function TDIDT( $S$ )  
  if  $imp(S) = 0$  then  
    return  $\langle \text{list: } S \rangle$   
  end if  
   $Test = optimal\_test(S)$   
   $t_L = TDIDT(S_L = \{e \in S : Test(e) = True\})$   
   $t_D = TDIDT(S_D = S \setminus S_L)$   
  return drevo( $\langle \text{vozlišče: } Test, \text{potomca: } t_L, t_D \rangle$ )  
end function
```

Kako izberemo optimalni test v notranjem vozlišču?

Množica možnih testov za diskretno spremenljivko X

Naj bo $D_X = \{x_1, x_2, \dots, x_m\}$ domena X

- Vsaka podmnožica $V \subset D_X$, $V \neq \emptyset$ definira en test
- Izpolnjen natanko takrat, ko je vrednost spremenljivke X element V
- Število možnih podmnožic V je $2^{|D_X|} - 2$
- Množici V in $D_X \setminus V$ definirata isti test
- Zato je število različnih testov $(2^{|D_X|} - 2)/2 = 2^{|D_X|-1} - 1$

Množica možnih testov za numerično spremenljivko X

Naj bo $D_X \subseteq \mathbb{R}$ domena X

- Vsak vrednost $v \in D_X, v \neq \min D_X$ določa en test $X < v$
- Število možnih testov je za eno manjše od števila vrednosti $v \in D_X$
- Število možnih testov je torej $|D_X| - 1$

Izbira optimalnega testa $\text{Test} = \text{optimal_test}(T)$

Ciljna funkcija za optimizacijo zmanjšanje nečistoče IR

$$IR(\text{Test}, T) = \text{imp}(T) - \left(\frac{|T_L|}{|T|} \text{imp}(T_L) + \frac{|T_D|}{|T|} \text{imp}(T_D) \right)$$

- Spomnimo se: test razbije T na disjunktni podmnožici T_L in T_D
- Pomen kratice IR je *Impurity Reduction* (zmanjševanje nečistoče)

Test izberemo z optimizacijo

$$\max_{\text{Test}} IR(\text{Test}, T) \equiv \min_{\text{Test}} \left(\frac{|T_L|}{|T|} \text{imp}(T_L) + \frac{|T_D|}{|T|} \text{imp}(T_D) \right)$$

Algoritem za izbiro testa za podano množico T

- 1 Naštejemo vse (ali naključno vzorčimo) teste Test
- 2 Izberemo tistega z največjo vrednostjo $IR(\text{Test}, T)$

Ali lahko prepoznam delfina?

primer	dolzina	skрге	kljun	zob	delfin
e ₁	3	ne	da	veliko	da
e ₂	4	ne	da	veliko	da
e ₃	3	ne	da	malo	da
e ₄	5	ne	da	veliko	da
e ₅	5	ne	da	malo	da
e ₆	5	da	da	veliko	ne
e ₇	4	da	da	veliko	ne
e ₈	5	da	ne	veliko	ne
e ₉	4	da	ne	veliko	ne
e ₁₀	4	ne	da	malo	ne

Opis naloge strojnega učenja

Primeri

10 opazovanih živali, od tega 5 delfinov

Spremenljivke

- Ciljna spremenljivka *deflin*, $D_{delfin} = \{da, ne\}$
- Numerična napovedna spremenljivka *dolzina*, $D_{dolzina} = \{2, 3, 4\} \subset \mathbb{R}$
- Diskretne napovedne spremenljivke
 - *skрге*, $D_{skрге} = \{da, ne\}$
 - *kljun*, $D_{kljun} = \{da, ne\}$
 - *zob*, $D_{zob} = \{veliko, malo\}$

Nečistost učne množice in možni testi

$$\text{imp}(S), S = \{e_1, e_2, \dots, e_{10}\}$$

- $P(\text{delfin} = da) = P(\text{delfin} = ne) = 5/10 = 0.5$
- $\text{Gini}(S) = 1 - (0.5^2 + 0.5^2) = 0.5$

Pet možnih testov

- 1 $\text{dolzina} < 4$
- 2 $\text{dolzina} < 5$
- 3 $\text{skрге} \in \{da\}$
- 4 $\text{kljun} \in \{da\}$
- 5 $\text{zob} \in \{\text{veliko}\}$

Test *dolzina* < 4

$$\text{imp}(S_L), S_L = \{e_1, e_3\}$$

- $P(\text{delfin} = da) = 2/2 = 1, P(\text{delfin} = ne) = 0/2 = 0$
- $\text{Gini}(S_L) = 1 - (1^2 + 0^2) = 0$

$$\text{imp}(S_D), S_D = \{e_2, e_4, e_5, e_6, e_7, e_8, e_9, e_{10}\}$$

- $P(\text{delfin} = da) = 3/8 = 0.375, P(\text{delfin} = ne) = 5/8 = 0.625$
- $\text{Gini}(S_D) = 1 - (0.375^2 + 0.625^2) = 0.46875$

$$IR(\text{dolzina} < 4) = 0.5 - \left(\frac{2}{10} 0 + \frac{8}{10} 0.46875 \right) = 0.125$$

Test *dolzina* < 5

$$\text{imp}(S_L), S_L = \{e_1, e_2, e_3, e_7, e_9, e_{10}\}$$

- $P(\text{delfin} = da) = 3/6 = 0.5, P(\text{delfin} = ne) = 3/6 = 0.5$
- $\text{Gini}(S_L) = 1 - (0.5^2 + 0.5^2) = 0.5$

$$\text{imp}(S_D), S_D = \{e_4, e_5, e_6, e_8\}$$

- $P(\text{delfin} = da) = 2/4 = 0.5, P(\text{delfin} = ne) = 2/4 = 0.5$
- $\text{Gini}(S_D) = 1 - (0.5^2 + 0.5^2) = 0.5$

$$\text{IR}(\text{dolzina} < 5) = 0.5 - \left(\frac{6}{10} 0.5 + \frac{4}{10} 0.5 \right) = 0$$

Test $skрге \in \{da\}$

$$imp(S_L), S_L = \{e_6, e_7, e_8, e_9\}$$

- $P(delfin = da) = 0/4 = 0, P(delfin = ne) = 4/4 = 1$
- $Gini(S_L) = 1 - (0^2 + 1^2) = 0$

$$imp(S_D), S_D = \{e_1, e_2, e_3, e_4, e_5, e_{10}\}$$

- $P(delfin = da) = 5/6 = 0.833, P(delfin = ne) = 1/6 = 0.167$
- $Gini(S_D) = 1 - (0.833^2 + 0.167^2) = 0.278$

$$IR(skрге \in \{da\}) = 0.5 - \left(\frac{4}{10}0 + \frac{6}{10}0.278\right) = 0.333$$

Test $kljun \in \{da\}$

$imp(S_L), S_L = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_{10}\}$

- $P(delfin = da) = 5/8 = 0.625, P(delfin = ne) = 3/8 = 0.375$
- $Gini(S_L) = 1 - (0.625^2 + 0.375^2) = 0.46875$

$imp(S_D), S_D = \{e_8, e_9\}$

- $P(delfin = da) = 0/2 = 0, P(delfin = ne) = 2/2 = 1$
- $Gini(S_D) = 1 - (0^2 + 1^2) = 0$

$$IR(kljun \in \{da\}) = 0.5 - \left(\frac{8}{10}0.46875 + \frac{2}{10}0\right) = 0.125$$

Test $zob \in \{veliko\}$

$imp(S_L), S_L = \{e_1, e_2, e_4, e_6, e_7, e_8, e_9\}$

- $P(delfin = da) = 3/7 = 0.4286, P(delfin = ne) = 4/7 = 0.5714$
- $Gini(S_L) = 1 - (0.4286^2 + 0.5714^2) = 0.4898$

$imp(S_D), S_D = \{e_3, e_5, e_{10}\}$

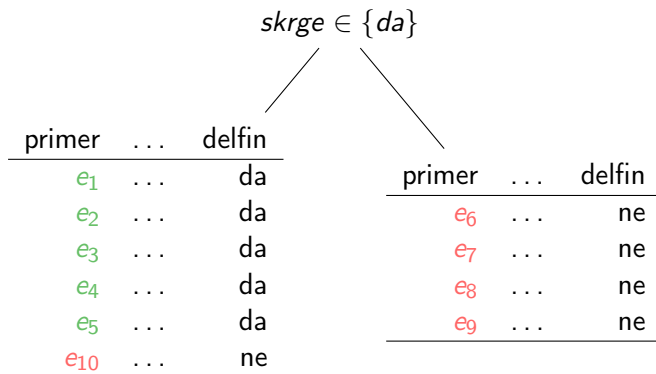
- $P(delfin = da) = 2/3 = 0.667, P(delfin = ne) = 1/3 = 0.333$
- $Gini(S_D) = 1 - (0.667^2 + 0.333^2) = 0.444$

$$IR(zob \in \{veliko\}) = 0.5 - \left(\frac{7}{10} 0.4898 + \frac{3}{10} 0.444 \right) = 0.0238$$

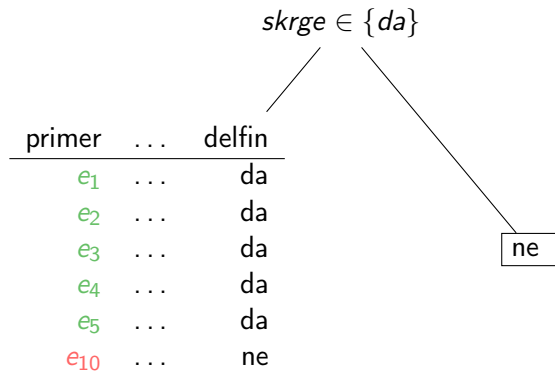
Izbira optimalnega testa: *skрге*

- ❶ $dolzina < 4$: 0.125
- ❷ $dolzina < 5$: 0
- ❸ $skрге \in \{da\}$: 0.333
- ❹ $kljun \in \{da\}$: 0.125
- ❺ $zob \in \{veliko\}$: 0.0238

Delni rezultat



Rekurzivno naprej.

Končno vozlišče, $Impurity = 0$ 

Rekurzija: nečistost množice in možni testi

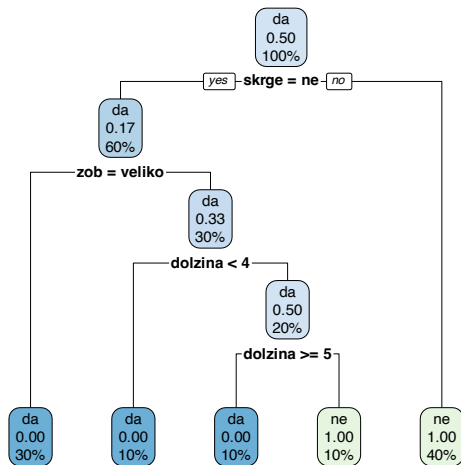
$imp(S), S = \{e_1, e_2, e_3, e_4, e_5, e_{10}\}$

- $p(delfin = da) = 5/6 = 0.833, p(delfin = ne) = 1/6 = 0.167$
- $Gini(S) = 1 - (0.833^2 + 0.167^2) = 0.278$

Štirje možni testi

- 1 $dolzina < 4$
- 2 $dolzina < 5$
- 3 $kljun \in \{da\}$
- 4 $zob \in \{veliko\}$

Končni rezultat



Kontrola pristranskosti in variance

Velikost drevesa

- Kontroliramo tako, da dopuščamo $Impurity > 0$ v končnih vozliščih
- Temu rečemo **rezanje dreves** (angl. *tree pruning*)

Sprotno in naknadno rezanje dreves

- Sprotno: določimo minimalno število primerov v listih
- Naknadno: spremenimo izbrana notranja vozlišča v liste

Naknadno rezanje dreves

Rezanje poddrevesa v vozlišču t

Notranje vozlišče t spremenimo v list t_K .

Definirajmo α -napako poddrevesa t

$$Error_{\alpha}(t) = Error(t) + \alpha|t|$$

- $Error(t)$: napaka poddrevesa v t na učnih primerih iz $S(t)$
- $|t|$: število končnih vozlišč v poddrevesu s korenem t

Vozlišče režemo, če velja $Error_{\alpha}(t_K) \leq Error_{\alpha}(t)$

Parameter α določa moč oz. stopnjo rezanja (večja vrednost, več rezanja).

Primer rezanja poddrevesa t : *dolzina* < 4

Napaka poddrevesa

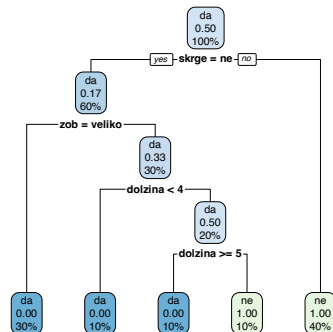
$Error_{\alpha}(t) = 0 + 3\alpha$, poddrevo ima napako 0 in tri končna vozlišča.

Napaka končnega vozlišča

$Error_{\alpha}(t_K) = 1/3 + 1\alpha$, učni primeri $\{e_3, e_5, e_{10}\}$: dva pozitivna in en negativen (napaka $1/3$).

Prelomna točka $Error_{\alpha}(t) = Error_{\alpha}(t_K)$

Če je $\alpha \geq 1/6$, potem bo poddrevo porezano; sicer pa ne.



Primer rezanja poddrevesa t : $zob \in \{veliko\}$

Napaka poddrevesa

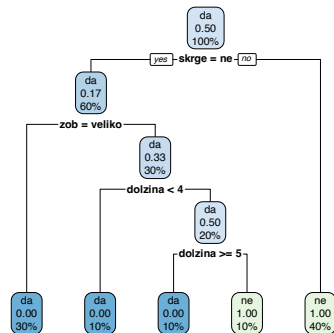
$Error_{\alpha}(t) = 0 + 4\alpha$, poddrevo ima napako 0 in štiri končna vozlišča.

Napaka končnega vozlišča

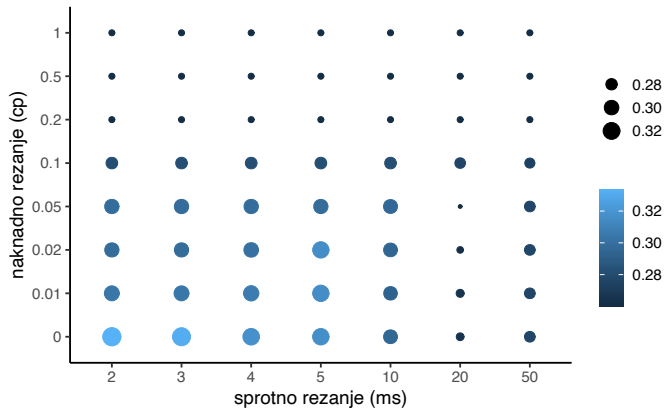
$Error_{\alpha}(t_K) = 1/6 + 1\alpha$, šest učnih primerov: pet pozitivnih in en negativen (napaka $1/6$).

Prelomna točka $Error_{\alpha}(t) = Error_{\alpha}(t_K)$

Če je $\alpha \geq 1/18$, potem bo poddrevo porezano; sicer pa ne.

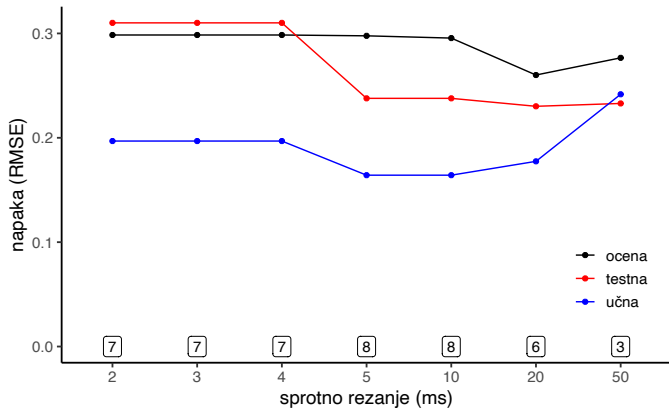


Regresijska drevesa: Sprotno in naknadno rezanje



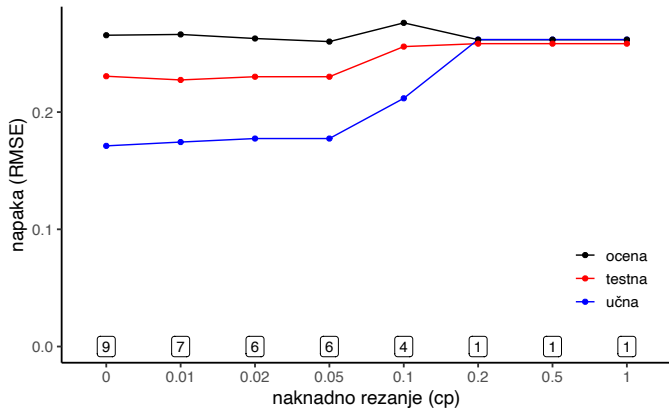
Regresijska drevesa ($\alpha = 0.05$): Sprotno rezanje

Oznake spodaj: število listov v drevesu.

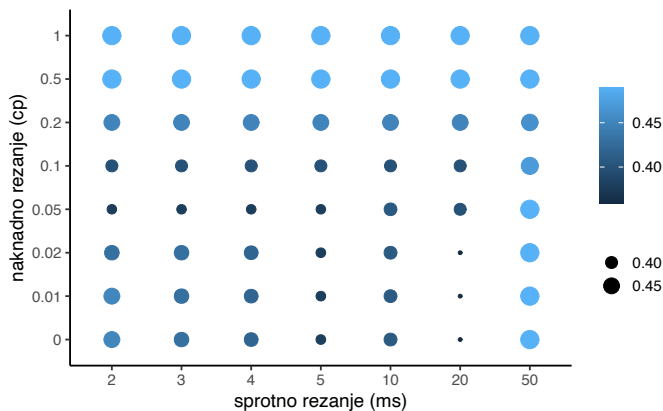


Regresijska drevesa ($ms = 20$): Izbira α

Oznake spodaj: število listov v drevesu.

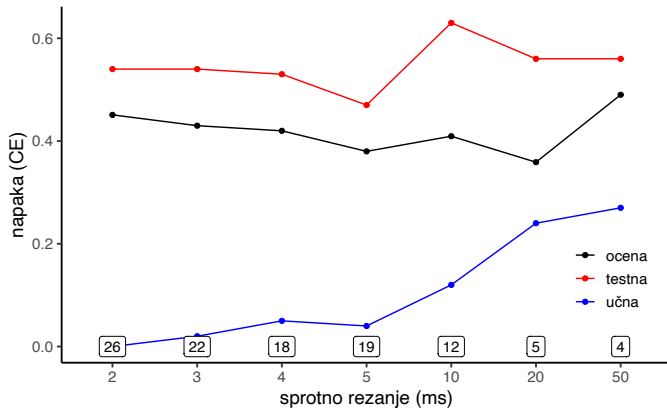


Klasifikacijska drevesa: Sprotno in naknadno rezanje



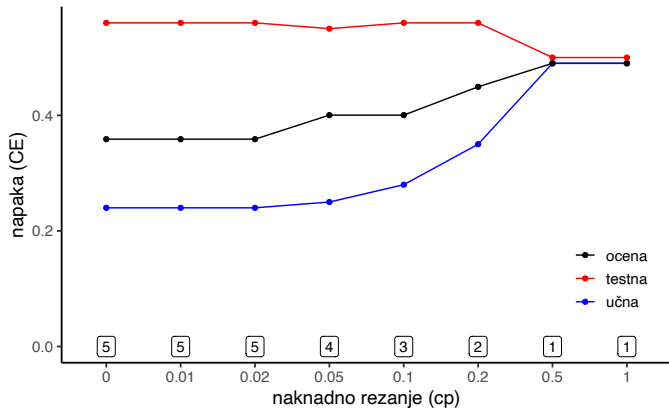
Klasifikacijska drevesa ($\alpha = 0.01$): Sprotno rezanje

Oznake spodaj: število listov v drevesu.



Klasifikacijska drevesa ($ms = 20$): Izbira α

Oznake spodaj: število listov v drevesu.



Odločitvena pravila

Kaj so odločitvena pravila?

Eno pravilo r : *IF* Pogoji *THEN* Napoved

- Pogoji je konjunkcija testov napovednih spr., npr. $X_1 > 3 \wedge X_3 = ne$
- Napoved poda napovedano vrednost ciljne spr. Y , npr. $\hat{Y} = da$

Urejena (tudi zveržena) množica pravil

Napoved za $\mathbf{x}$ poda prvo pravilo r , za katerega velja: $\text{Pogoj}(\mathbf{x}) = \text{True}$.

Neurejena množica pravil

- Napovedi za $\mathbf{x}$ podajo vsa pravila r : $\text{Pogoj}(\mathbf{x}) = \text{True}$
- Povprečje napovedi (regresija) ali večinsko glasovanje (klasifikacija)

Primeri množice pravil

Urejena množica pravil

IF skрге = da THEN delfin = ne ELSE
IF zob = veliko THEN delfin = da ELSE
IF dolzina = 4 THEN delfin = ne ELSE
delfin = da

Neurejena množica pravil

IF dolzina = 3 THEN delfin = da
IF skрге = ne $\wedge$ zob = veliko THEN delfin = da

Implementacija v scikit-learn

Odločitvena drevesa

- Modul `sklearn.tree`
- Klasifikacija, razred `DecisionTreeClassifier`
- Regresija, razred `DecisionTreeRegressor`

Učenje enega pravila

Require: S je množica učnih primerov

Ensure: r je naučeno odločitveno pravilo

```
function Pravilo( $S$ )  
  Pogoj = True  
  while  $imp(S, Pogoj) > 0$  do  
     $L_{opt} = \arg \min_L imp(S, Pogoj \wedge L)$   
    Pogoj =  $Pogoj \wedge L_{opt}$   
  end while  
  return  $r = IF \text{ Pogoj } THEN Y = Prediction(S, Pogoj)$   
end function
```

Napoved pravila $Prediction(S, \text{Pogoj})$

$$S_{\text{Pogoj}} = \{\langle \mathbf{x}, y \rangle \in S : \text{Pogoj}(\mathbf{x}) = \text{True}\}$$

Regresija, $D_Y \subseteq \mathbb{R}$: povprečje

$$Prediction(S, \text{Pogoj}) = \frac{1}{|S|} \sum_{\langle \mathbf{x}, y \rangle \in S_{\text{Pogoj}}} y$$

Klasifikacija, $D_Y = \{y_1, y_2, \dots, y_c\}$: večinski razred

$$Prediction(S, \text{Pogoj}) = \arg \max_{y_i \in D_Y} P(Y = y_i | S_{\text{Pogoj}})$$

Funkcija nečistoče $imp(S, \text{Pogoj})$

Funkcija nečistoče meri varianco vrednosti ciljne spremenljivke Y v množici **pokritih** primerov $S_{\text{Pogoj}} = \{\langle \mathbf{x}, y \rangle \in S : \text{Pogoj}(\mathbf{x}) = \text{True}\}$.

Regresija, $D_Y \subseteq \mathbb{R}$

$$imp(S, \text{Pogoj}) = \frac{1}{|S|} \sum_{\langle \mathbf{x}, y \rangle \in S_{\text{Pogoj}}} (y - \bar{y})^2$$

Klasifikacija, $D_Y = \{y_1, y_2, \dots, y_c\}$

$$imp(S, \text{Pogoj}) = 1 - \max_{v \in D_Y} P(Y = v | S_{\text{Pogoj}})$$

Učenje urejenih pravil: prekrivni algoritem

Require: S je množica učnih primerov

Ensure: R je seznam urejenih odločitvenih pravil

```
function Pravila( $S$ )  
   $R = []$   
  while  $S \neq \emptyset$  do  
     $r = \text{Pravilo}(S)$   
     $S = S \setminus S_{\text{Pogoj}(r)}$   
  end while  
  return  $R$   
end function
```

Učenje neurejenih pravil: Spremembe algoritma

- V vsaki iteraciji izberemo razred $y_i \in D_Y$ za katerega se učimo pravilo
- Posebna varianta $LearnRule(S, y_i)$, ki se uči pravilo za izbran razred y_i
- Iz množice S brišemo le primere iz razreda y_i : $S = S \setminus S_{\text{Pogoj}(r) \wedge Y=y_i}$

Ali lahko prepoznam delfina?

primer	dolzina	skрге	kljun	zob	delfin
e ₁	3	ne	da	veliko	da
e ₂	4	ne	da	veliko	da
e ₃	3	ne	da	malo	da
e ₄	5	ne	da	veliko	da
e ₅	5	ne	da	malo	da
e ₆	5	da	da	veliko	ne
e ₇	4	da	da	veliko	ne
e ₈	5	da	ne	veliko	ne
e ₉	4	da	ne	veliko	ne
e ₁₀	4	ne	da	malo	ne

Prva iteracija algoritma Pravilo(S)

Na začetku $Pogoj = True$, $S = \{e_1, e_2, \dots, e_{10}\}$

L	S_{Pogoj}	p_{da}	p_{ne}	$imp(S, Pogoj)$
$dolzina = 3$	$\{e_1, e_3\}$	1	0	0
$dolzina = 4$	$\{e_2, e_6, e_9, e_{10}\}$	0.25	0.75	0.25
$dolzina = 5$	$\{e_4, e_5, e_8, e_{10}\}$	0.5	0.5	0.5
$skрге = da$	$\{e_6, e_7, e_8, e_9\}$	0	1	0
$skрге = ne$	$\{e_1, \dots, e_5, e_{10}\}$	0.83	0.17	0.17
$kljun = da$	$\{e_1, \dots, e_7, e_{10}\}$	0.625	0.375	0.375
$kljun = ne$	$\{e_8, e_9\}$	0	1	0
$zob = veliko$	$\{e_1, e_2, e_4, e_6, \dots, e_9\}$	0.43	0.57	0.43
$zob = malo$	$\{e_3, e_5, e_{10}\}$	0.67	0.33	0.33

Prva iteracija algoritma Pravila(S)

Izbrano pravilo *IF skрге = da THEN delfin = ne*

- Ker od vseh pravil z *Impurity* = 0 pokrije največ primerov (4)
- Ostane torej šest primerov $S = \{e_1, \dots, e_5, e_{10}\}$

Prva iteracija algoritma Pravila(S), drugič

Na začetku $Pogoj = True$, $S = \{e_1, \dots, e_5, e_{10}\}$

L	S_{Pogoj}	p_{da}	p_{ne}	$imp(S, Pogoj)$
$dolzina = 3$	$\{e_1, e_3\}$	1	0	0
$dolzina = 4$	$\{e_2, e_{10}\}$	0.5	0.5	0.5
$dolzina = 5$	$\{e_4, e_5\}$	1	0	0
$kljun = da$	$\{e_1, \dots, e_5, e_{10}\}$	0.83	0.17	0.17
$kljun = ne$	$\emptyset$	NaN	NaN	NaN
$zob = veliko$	$\{e_1, e_2, e_4\}$	1	0	0
$zob = malo$	$\{e_3, e_5, e_{10}\}$	0.67	0.33	0.33

Druga iteracija algoritma Pravila(S)

Izbrano pravilo *IF zob = veliko THEN delfin = da*

- Pokriva tri primere
- Ostanejo trije nepokriti primeri $S = \{e_3, e_5, e_{10}\}$

Trenutna (urejena) množica pravil

IF skрге = da THEN delfin = ne ELSE
IF zob = veliko THEN delfin = da

Prva iteracija algoritma Pravilo(S), tretjič

Na začetku $Pogoj = True$, $S = \{e_3, e_5, e_{10}\}$

L	S_{Pogoj}	p_{da}	p_{ne}	$imp(S, Pogoj)$
$dolzina = 3$	$\{e_3\}$	1	0	0
$dolzina = 4$	$\{e_{10}\}$	0	1	0
$dolzina = 5$	$\{e_5\}$	1	0	0
$kljun = da$	$\{e_3, e_5, e_{10}\}$	0.67	0.33	0.33
$kljun = ne$	$\emptyset$	NaN	NaN	NaN

Tretja iteracija algoritma Pravila(S)

Izbrano pravilo *IF dolzina = 4 THEN delfin = ne*

- Pokriva en primer
- Ostaneta dva pozitivna primera $S = \{e_3, e_5\}$, čista množica

Končna (urejena) množica pravil

IF skрге = da THEN delfin = ne ELSE
IF zob = veliko THEN delfin = da ELSE
IF dolzina = 4 THEN delfin = ne ELSE
delfin = da

Kontrola pristranskosti in variance

Velikost množice pravil

- Število pravil v množici
- Število testov v (vseh) pravilih

Kontrola velikosti

- 1 Sprotno rezanje: predčasni izhod iz iteracije algoritma Pravilo
- 2 Naknadno rezanje: odstranjevanje pravil ali posameznih testov

Znani algoritmi in implementacije za drevesa

CN2 (Clark and Nibblet 1987)

Funkcija `RI.CN2Rules.RST` v R paketu `RoughSets`.

Ripper (Cohen 1995)

Na voljo v zbirki Weka, v R-ju skozi paket `RWeka`.

Znani algoritmi in implementacije za pravila

CART (Breiman in ost. 1984)

Paket rpart v R-ju.

C4.5 (Quinlan 1995), pozneje C5.0

Na voljo v zbirki Weka kot J4.8 (C4.5), tudi paket za R (C5.0).